



Ternary Bonsai 2 27B

A 27B ternary reasoning model.

~9.1×

smaller

98.2%

intelligence retained

46.8 tok/s

on laptop

Contents

1	Executive Summary	2
2	Ternary Bonsai 2 27B Model Summary	3
2.1	Ternary Weight Format	3
2.2	Select full-precision tensors	3
2.3	Storage Footprint	4
2.4	Rotated Weight Basis	5
3	Inference: Low-Bit Kernels for Hybrid-Attention Models	5
4	Benchmarks	6
5	Limitations and Roadmap	8
A	Deployment and Kernel Reference	9
A.1	Backend Implementation	9
A.2	Executing the Rotated Basis	9
B	Benchmark Evaluation Methodology	9
C	All Benchmark Results	11
D	Throughput and Energy Measurement Notes	12

1. Executive Summary

Bonsai 2 27B uses the same hybrid-attention architecture as Qwen3.8-27B [3]. Its matrix weights are ternary in a fixed rotated basis, taking values in $\{-1, 0, +1\}$ with FP16 group scales. This gives an effective footprint of 1.76 bits per weight as shipped, or **5.93 GB**, compared with **53.8 GB** in FP16.

Conventional low-bit quantization methods fail qualitatively and unevenly. Long context reasoning and coding deteriorate first, while surface fluency and short tool calls retain high performance. As such, the failure is easy to miss in casual testing and expensive to discover in production.

Ternary Bonsai 2 27B averages 83.9 on the 20-benchmark suite of Section 4, retaining **98.2%** of the **Qwen3.8-27B** FP16 baseline's 85.4 and **slightly exceeding** the 83.6 average of **Qwen3.6-27B**, at roughly a ninth of their size. The suite spans reasoning, math, coding, instruction following, tool calling and vision.

This release also delivers on the roadmap set out in our initial Bonsai 27B release [2], where we identified long-horizon, tool-driven software engineering as a major next step. On TerminalBench 2.1, Ternary Bonsai 2 27B demonstrates strong capability at this footprint despite aggressive compression. See Section 4 for details.

2. Ternary Bonsai 2 27B Model Summary

Ternary Bonsai 2 27B can be summarized by the following table. To be explicit about scope, Bonsai applies its ternary representation end-to-end across almost all of the *language* model's parameters. The vision tower is packaged separately. The GGUF release uses a 4-bit representation, while the MLX package retains it in full precision. Within the language model, 0.0976% of parameters are held above the low-bit representation, and that is counted in the advertised 1.76 bits per weight budget.

Table 1. System specification.

Item	Specification
Architecture model	Based on Qwen3.8-27B [3], a 27B hybrid-attention causal language model
Parameters	24.35B language (64 blocks) + 0.47B vision tower (27 blocks) + 2.54B embedding/lm-head = 27.36B total
Architecture	Hybrid attention (~75% linear attention / ~25% full attention), SwiGLU MLP, RoPE, RMSNorm
Context length	262K tokens (full-context capable, enabled by the predominantly linear-attention backbone)
Weight format	Ternary g128 (FP16 group-wise scaling) → One fp16 scale for 128 ternary elements
Reasoning Efforts	xhigh or medium (Low does not reduce thinking)
Weight basis	Blockwise Hadamard rotation (block 1024, fixed ± 1 signs)
Low-bit weights	Embeddings, attention and linear-attention projections, MLP projections, LM head
Backends	MLX (Python, Swift) and CUDA
License	Apache License

2.1 Ternary Weight Format

Each weight takes a value from $\{-1, 0, +1\}$, with one shared FP16 scale for every group of 128 weights. The ternary weight in the rotated basis is

$$w_i = s_g \cdot t_i, \quad t_i \in \{-1, 0, +1\},$$

where s_g is the scale for group g . A ternary value carries $\log_2 3 \approx 1.585$ bits of information, so with one FP16 scale per 128 weights the effective storage cost is

$$b_{\text{eff}} \approx \log_2 3 + \frac{16}{128} \approx 1.71 \text{ bits/weight},$$

That rate covers the ternary tensors only. Including the high-precision tensors of Section 2.2, the figure tabulated over all language-model parameters in Table 3 is 1.72 bits per weight, a reduction of $\approx 9.3\times$ relative to FP16.

2.2 Select full-precision tensors

Ternary Bonsai 2 27B holds a small set of tensors above the low-bit representation, while the previous release ternarized these alongside everything else. They are neither rotated nor quantized, and are listed in the table below. The additional cost of this choice is 26.2 M parameters, which constitutes only 0.0976% of the language model, or 52 MB at bf16. It moves the ternary model from 1.71 to 1.72 bits per weight.

Table 2. Tensors kept in full precision.

Tensor group	Shape	Layers	Params	Share
<i>Recurrent state path (linear-attention layers)</i>				
in_proj_a.weight	5120 × 48	48	11,796,480	0.0439%
in_proj_b.weight	5120 × 48	48	11,796,480	0.0439%
conv1d.weight	10240 × 4	48	1,966,080	0.0073%
A_log, dt_bias, norm.weight	48, 48, 128	48	10,752	0.00004%
<i>Normalization</i>				
input_/post_attention_layernorm, final	5120	64, 1	660,480	0.0025%
q_norm, k_norm	256	16	8,192	0.00003%
Total high precision			26,238,464	0.0976%

2.3 Storage Footprint

While the weights of Bonsai 2 27B can be represented using 1.72 bits per weight in principle, practical deployment requires packing formats that support efficient inference kernels. We implement two such formats, **PTQ1_0** and **PQ2_0**. PTQ1_0 has smaller footprint, while PQ2_0 uses a simpler 2-bit representation that is cheaper to unpack. Neither format is uniformly faster. Section 3 compares their throughput.

Table 3. Storage footprint of the language model. Sizes are decimal GB.

Format	bpw	Size	Reduction
FP16 (baseline)	16.0	53.80 GB	1.0×
True Ternary	1.72	5.80 GB	~9.3×
PTQ1_0	1.76	5.93 GB	~9.1×
PQ2_0	2.16	7.25 GB	~7.4×

Vision tower. The figures above refer to the language model alone, which is the only component required for text-only inference. The vision tower is an optional component used only for multimodal inputs.

Table 4. Shipped components of the GGUF releases. Sizes are on disk.

Component	Pack	Size	Use
Language model	ternary g128 (PTQ1_0)	5.93 GB	text and multimodal inference
Language model	ternary g128 (PQ2_0)	7.25 GB	text and multimodal inference
Vision tower	mmproj HQQ 4-bit (Q8_0 container)	0.63 GB	multimodal input only
Vision tower	mmproj BF16 (reference)	0.93 GB	multimodal input only

MLX packaging. The MLX release uses a 2-bit, group-size-128 packing analogous to PQ2_0, but its block format stores both an FP16 scale and an FP16 bias per group. Bonsai's ternary weights require only a scale: the bias is redundant and could be omitted. As a result, the MLX block ~~costs 36~~ rather than 34 bytes per 128 weights, increasing the packed rate from 2.125 to 2.250 bits per weight. Together with the full-precision vision tower and the small high-precision state path, this gives the published 8.49 GB package.

④ Bytes → $128 \times 2 = 256 \text{ bits} = 32 \text{ bytes}$
 + 2 bytes per FP16 ⇒ 36 bytes

2.4 Rotated Weight Basis

Ternary Bonsai 2 27B uses its weights with a fixed rotation. Ternary matrix weights W are stored in memory and then transformed blockwise by

$$R = \frac{1}{\sqrt{n}} H_n S, \quad n = 1024,$$

where H_n is a Walsh–Hadamard matrix and S a fixed diagonal of ± 1 signs [28]. Mathematically, inference at each layer is performed as follows, where x is the output of the previous layer:

$$f(x) = W(Rx)$$

3. Inference: Low-Bit Kernels for Hybrid-Attention Models

A compact weight representation is useful only if the runtime can execute it directly. Existing low-bit inference paths primarily target standard full-attention transformers and do not cover the hybrid-attention structure of Qwen3.8-27B, which interleaves linear- and full-attention layers. Ternary Bonsai 2 27B therefore uses custom kernels for this hybrid backbone on Apple Silicon and CUDA. The kernels operate directly on the packed ternary weights, avoiding materialization of dense FP16 weights. This is essential to realizing the representation’s memory-bandwidth advantage during inference. Implementation details for both backends are given in Appendix A.

The rotated representation requires an activation-side Walsh–Hadamard transform at runtime. Its implementation, optimization, and backend-specific overhead are described in Appendix A.

Cross-Platform Throughput We report throughput with the standardized $\text{tg}_{128}/\text{pp}_{512}$ measurements defined in Appendix D: tg_{128} is token-generation throughput over 128 generated tokens and pp_{512} is prompt-processing throughput over 512 input tokens, both in tokens per second. On the edge platforms, such as a laptop with 16GB of RAM, the full-precision baseline (54 GB) and even Q4_K_XL [8] (17.6 GB) do not fit. In contrast, both packings **PTQ1_0** and **PQ2_0** of Bonsai 2 27B defined in Section 2.3 fit and Table 5 reports throughputs for them.

Table 5. Cross-platform throughput and energy for the two Ternary Bonsai 2 27B packing formats.

Hardware	PQ2_0			PTQ1_0		
	TG128 (tok/s)	PP512 (tok/s)	mWh/tok	TG128 (tok/s)	PP512 (tok/s)	mWh/tok
RTX 5090 (32 GB)	142.5	4121	0.582	134.4	1901	0.609
RTX PRO 6000 Blackwell	140.6	4520	0.637	136.8	2290	0.642
H200 SXM (141 GB)	118.0	2818	0.708	89.5	1216	0.814
B200 (180 GB)	117.1	3112	0.830	91.9	1406	1.019
H100 NVL (94 GB)	106.4	2484	0.541	81.6	1098	0.647
H100 SXM (80 GB)	103.2	2467	0.584	77.3	1097	0.706
RTX 4090 (24 GB)	90.9	3134	0.714	96.7	1634	0.682
RTX 6000 Ada (48 GB)	84.8	2430	0.731	92.0	1627	0.701
L40S (48 GB)	74.6	2827	0.812	82.8	1601	0.743
A100 SXM (80 GB)	74.0	1328	0.776	54.6	703	0.932
L4 (24 GB, 72 W)	29.7	778	0.629	32.1	468	0.585
Laptop (M5 Pro, Metal)	27.7	397	—	27.1	369	—

The footprint advantage is most visible on edge hardware. On the M5 Pro, the ternary model sustains 27.7 tok/s, corresponding to ≈ 201 GB/s of weight traffic and indicating a decode path dominated by memory bandwidth. On an RTX 5090, the same model reaches 142.5 tok/s at 0.582 mWh/token.

The Apple row does not report energy per token because power measurements are not directly comparable across platforms. NVIDIA board power includes HBM/GDDR power, whereas Apple powermetrics does not include DRAM. Since batch-1 decoding is largely memory-bandwidth-bound, excluding memory power would systematically bias the Apple energy figure downward. We therefore report measured power directly rather than deriving a cross-platform energy-per-token comparison.

For sustained plain-autoregressive decoding of the PQ2_0 pack on the M5 Pro, measured at 27.0 tok/s over a longer window than the `tg128` measurement in Table 5, powermetrics reports 27.0 W on the GPU rail and 32.8 W across CPU and GPU. The corresponding idle floor is 0.17 W for the GPU and 1.47 W for the CPU. We report these values separately rather than subtracting them. Additional Apple Silicon measurements are given in Table 6. The measurement protocol and architecture-specific throughput discussion are provided in Appendix D.

Table 6. Ternary Bonsai 2 27B throughput across Apple Silicon laptops using the `llama.cpp` Metal backend and PQ2_0 packing.

Platform	TG128 (tok/s)	PP512 (tok/s)
Laptop (M5 Max)	46.8	765
Laptop (M5 Pro)	27.7	397
Laptop (M4 Pro)	18.0	125

On the wider M5 Max, the ternary model reaches approximately 47 tok/s in batch-1 decoding. On the M4 Pro, decode remains interactive at approximately 18 tok/s, while prompt processing at approximately 125 tok/s becomes the more relevant constraint for very long contexts. The M4 Pro figures come from the earlier pre-rotation build and should therefore be interpreted primarily as indicators of scaling across Apple Silicon rather than as direct release-stack comparisons.

The practical takeaway is not simply the throughput itself, but what it enables: interactive, fully on-device generation from a 27B-class model on an everyday laptop.

4. Benchmarks

We evaluate all models in thinking mode using EvalScope [4] with the vLLM [5] backend on NVIDIA H100 GPUs. The evaluation suite contains 20 benchmarks spanning knowledge and reasoning, math, coding, instruction following, agentic tool use, and vision. Category-level averages are reported in Table 8.

All results in the present section refer to the `xhigh` reasoning effort. Full per-benchmark results at both `xhigh` and `medium` reasoning effort are reported in Appendix C.

At 5.93 GB, Ternary Bonsai 2 27B achieves an average score of 83.9, retaining 98.2% of the Qwen3.8-27B FP16 baseline’s 85.4. Its average is also slightly above the 83.6 obtained by Qwen3.6-27B. The conventional IQ2_XXS [7] build achieves 75.2 at 7.3 GB, despite being $1.23\times$ larger.

Table 7. Thinking-mode average across 20 benchmarks. Bpw is the average bit-width over all model weights for the packed checkpoint.

Variant	True bpw	Footprint	Thinking avg	vs Qwen3.8 FP16
Qwen3.8-27B FP16	16.0	53.80 GB	85.4	100%
Qwen3.6-27B FP16	16.0	53.80 GB	83.6	97.9%
Qwen3.8-27B IQ2_XXS [7]	2.2	7.3 GB	75.2	88.4%
Ternary Bonsai 2 27B	1.76	5.93 GB	83.9	98.2%

The aggregate average understates where quality is preserved. IQ2_XXS degrades sharply on several reasoning-intensive tasks, falling, for example, to 78.6 on AIME26 and 70.05 on Live-CodeBench. Ternary Bonsai scores 95.83 and 90.07 on the same benchmarks.

More importantly, Bonsai holds full-precision quality on the capabilities that matter most for reasoning and agentic use. Math reaches 96.57 compared with 97.06 for Qwen3.8 and 94.64 for Qwen3.6, while coding reaches 81.58 compared with 82.17 and 82.57. Instruction following improves to 82.66, above both Qwen3.8 at 81.25 and Qwen3.6 at 74.53. Agentic and tool-calling performance holds at 77.57, compared with 79.74 for Qwen3.8 and 80.05 for Qwen3.6.

Long-context and coding performance. This release also delivers on the roadmap set out in our initial Bonsai 27B release [2], where we identified long-horizon, tool-driven software engineering as the next major capability to improve. With Ternary Bonsai 2 27B, that progress now shows up directly in agentic performance. Evaluated for the first time on Terminal-Bench 2.1 and SWE-bench Verified, the Ternary Bonsai 2 27B reaches 52.8 and 60.8, respectively, compared with 69.7 and 80.6 for Qwen3.8-27B, retaining roughly three quarters of the full-precision performance on both benchmarks.

This is particularly notable because long-horizon agentic evaluations are unforgiving under aggressive compression: models at this footprint often fail to make any meaningful progress, while conventional low-bit quantization can cause agentic performance to collapse. Ternary Bonsai 2 27B instead remains capable of sustained tool use and end-to-end software-engineering workflows. The gains extend across the broader agentic and long-context reasoning tasks as well: on τ^2 -Bench, our new model reaches 80.2, up from 73.6 in the previous release; BFCL v3 holds at 74.9; and AA-LCR remains within a point of full precision at 77.0.

Together with the gains in coding, instruction following, and reasoning, these results move Bonsai beyond compact chat models. A roughly 6 GB model can now run locally as the language model for coding agents, sustaining long-horizon tool use and completing end-to-end software-engineering tasks.

Table 8. Thinking-mode scores by skill category (category averages over the benchmarks listed).

Category	Benchmarks	Qwen3.8 27B	Ternary 27B	Qwen3.6 27B
Agentic / tool calling	τ^2 -Bench, BFCL v3	79.74	77.57	80.05
Coding	HumanEval+, LiveCodeBench v6, MBPP+, BigCodeBench	82.17	81.58	82.57
Math	AIME26, AIME25, GSM8K, MATH-500	97.06	96.57	94.64
Knowledge & reasoning	MMLU-Redux, GPQA Diamond, AA-LCR	86.66	83.95	84.71
Instruction following	IFBench, IFEval	81.25	82.66	74.53
Vision	CharXiv, A-OKVQA, OmniDocBench v1.6, RealWorldQA, OCRBench v2	81.64	78.59	79.82
Overall (20)		85.4	83.9	83.6

Intelligence Density Following the framework introduced in the 1-bit Bonsai report [1], *intelligence density* measures benchmark performance relative to storage footprint. We denote the average score on the 20-benchmark suite by avg and define the corresponding error probability as $P_e = 1 - \text{avg}/100$. Intelligence is then defined as $-\log_2 P_e$, and intelligence density as

$$ID = \frac{-\log_2 P_e}{N},$$

for a model occupying N gigabytes.

Table 9. Intelligence density (1/GB), thinking mode, on the 20-benchmark suite.

Variant	Size (GB)	Avg	Density (1/GB)
Ternary Bonsai 2 27B	5.93	83.9	0.444
Qwen3.8-27B IQ2_XXS	7.3	75.2	0.276
Qwen3.8-27B FP16	53.80	85.4	0.051

Intelligence density combines benchmark performance and storage footprint into a single quantity. The logarithmic error term places greater weight on preserving high accuracy, while division by model size favors compact representations. On the 20-benchmark suite, Ternary Bonsai 2 27B attains a density of 0.444 1/GB, which is substantially above the intelligence densities of 0.051 for Qwen 3.8 - 27B and 0.276 for IQ2_XXS.

5. Limitations and Roadmap

Improving the native low-bit kernels. PTQ1_0 stores weights at 1.76 bits per weight, approaching the 1.72 bits per weight limit possible for the chosen ternary format. PQ2_0 uses 2.16 bits per weight. As Table 5 shows, PTQ1_0 is faster on Ada-class parts but slower on Ampere, Hopper, Blackwell and Apple silicon. Making PTQ1_0 as fast as PQ2_0 in all scenarios in addition to its lower footprint is hence an active engineering target.

KV compression (sub-4-bit). Early results show that KV cache can be pushed toward the sub-4-bit regime, while retaining benchmark performance. This would open a path to still longer contexts within a fixed device-memory budget.

Rotation overhead. At batch size 1 the activation-side rotation transform remains one of the larger non-matrix-multiply costs of a decode step. This is a pure implementation cost with no bearing on output quality, and it is the most direct remaining lever on on-device decode latency.

A. Deployment and Kernel Reference

A.1 Backend Implementation

Apple Silicon (MLX [6]). The ternary path uses packed low-bit weights with per-group FP16 scales and custom kernels for the hybrid-attention stack. Both Python and Swift runtimes use the same packed representation.

CUDA. The CUDA backend uses fused low-bit GEMM kernels that unpack ternary codes and apply group-wise scales inside the matrix multiply. Activations and numerically sensitive operations remain in higher precision.

A.2 Executing the Rotated Basis

Before each folded matrix multiply, the activation is transformed by a fixed sign flip followed by a block-wise fast Walsh-Hadamard transform. The Hadamard-Walsh transform is $O(n \log n)$ in the activation length [29], which is negligible compared to the matrix-multiply it precedes.

At batch size 1, this transform lies on the critical path of every projection. On Metal, the sign flip and transform account for a substantial fraction of the operations dispatched per generated token. We therefore fuse the sign flip into the transform's load path, eliminating a separate activation pass.

On CUDA, the transform is parallelized across a full thread block rather than a single warp, reducing serialization across the Hadamard stages. This substantially reduces the transform's share of decode time.

Prompt processing is less sensitive to this overhead because the transform is amortized across many input tokens and the matrix multiplication itself is more compute-bound.

B. Benchmark Evaluation Methodology

All models are evaluated with the same EvalScope [4] harness and served through vLLM [5] on NVIDIA H100 GPUs. Full-precision baselines, conventional low-bit references, and Bonsai models use the same dataset versions, serving path, and scoring pipeline.

Results are reported in *thinking* mode throughout, with a server-side reasoning parser so that extractors and judges score only the final answer. Generation is sampled at top- p 0.95 and top- k 20, with temperature at each model's recommended setting. Scoring is rule-based wherever possible: sandboxed execution for code (pass@1), programmatic verifiers for instruction following, parsing and state checks for tool calling, reference matching for OCR, and a rule-first extractor with a fixed LLM-judge fallback (Gemini 3 Flash, identical across models) for knowledge, reasoning, and math. Most benchmarks are single-pass. AIME25/26 report mean accuracy over 8 samples. Text benchmarks are served language-model-only. The vision benchmarks are served with the vision tower loaded.

The 20-benchmark suite comprises: MMLU-Redux [9], GPQA Diamond [10], and AA-LCR [11] for knowledge and reasoning; GSM8K [12], MATH-500 [13], AIME25 [14], and AIME26 [15] for math; HumanEval+ and MBPP+ [16], LiveCodeBench v6 [17], and BigCodeBench [18] for coding; IFEval [19] and IFBench [20] for instruction following; τ^2 -Bench [21] and BFCL v3 [22] for agentic and tool-calling performance; and CharXiv [23], A-OKVQA [24], OmniDocBench v1.6 [25], RealWorldQA [26], and OCR Bench v2 [27] for vision. The corresponding per-benchmark scores are reported in Appendix C. Moreover, we evaluate Ternary Bonsai 2 27B and Qwen 3.8 27B on TerminalBench v2.1 [32] and Swe-Bench-Verified [33].

Evaluation protocol and grading choices. All 20 benchmarks are run through the same EvalScope-based harness against a vLLM server (tensor-parallel 2, data-parallel 4), in thinking mode with reasoning effort x_{high} and the sampling recommended by the Qwen3.8 model card (temperature 1.0, top- p 0.95,

top- k 20, no presence or repetition penalty). Every item is answered once (single-sample pass@1 at temperature 1, not greedy decoding), the reasoning block is stripped before grading, and generation budgets are generous: 81,920 output tokens for AIME, MATH-500, GPQA Diamond, LiveCodeBench and IF-Bench; 49,152 for HumanEval+, BigCodeBench, OmniDocBench, OCRBench and τ^2 -Bench; 32,768 for the remaining benchmarks. Where a benchmark offers several variants we made the following choices. MMLU-Redux uses the 2.0 release (57 subjects, 5,700 questions), zero-shot with an answer-line extraction. GPQA Diamond is sampled five times per question (990 completions) and we report mean accuracy, not majority vote. AIME25 and AIME26 are sampled eight times per problem (240 completions each) and we report mean accuracy (avg@8), not pass@ k . GSM8K and MATH-500 are zero-shot chain-of-thought with numeric equivalence on the boxed answer; MATH-500, GPQA, AIME and AA-LCR fall back to an LLM judge (*gemini-3-flash-preview*, temperature 0) when rule-based matching is inconclusive. HumanEval+ and MBPP+ use the EvalPlus extended test suites (164 and 378 problems). LiveCodeBench uses the *release_v6* problem set (1,055 problems) graded by unit tests. BigCodeBench uses the *complete* split (docstring-based prompts, not the *instruct* split) over the full 1,140 problems rather than the 148-problem Hard subset, version 0.1.4 with calibrated pass@1 in a Docker sandbox. For IFEval and IFBench we report prompt-level strict accuracy, the most demanding of the four standard metrics. τ^2 -Bench covers the airline, retail and telecom domains (278 tasks) and is scored as pass@1 from a single trial, with *deepseek-v4-flash* at temperature 0 as the user simulator. BFCL v3 is run in prompt-based function-calling mode (tool schemas in the system prompt, calls parsed from text rather than native tool-call messages) over all 17 subsets including multi-turn (4,441 items), and we report the sample-weighted mean accuracy over all items. CharXiv covers both the descriptive and reasoning splits (5,000 questions); A-OKVQA uses the validation split (1,145 multiple-choice questions); OmniDocBench v1.6 reports the benchmark’s *overall* composite of edit-distance, CDM and TEDS sub-scores; RealWorldQA uses the 765-item test set; OCRBench v2 covers all 30 English and Chinese subsets (10,000 items) with mean accuracy.

For the agentic benchmarks all models are served identically and driven by the Harbor framework. Terminal-Bench 2.1 uses the Terminus-2 agent on all 89 tasks with a single attempt per task, eight concurrent trials, 32 CPUs and 48 GB of memory per task container, a 262,144-token context and an 80,000-token output budget per turn. Terminus-2 supports a JSON and an XML action format; the JSON parser is the harness default and is what we report, but XML variant performs similarly. SWE-bench Verified uses the mini-swe-agent scaffold on all 500 instances with the same sampling and time-limit settings, and we report the resolved rate.

C. All Benchmark Results

Table 10. Full per-benchmark results (%), thinking mode with reasoning-effort xhigh. Sizes are language-model weights in decimal GB. Bit-widths are averages over language-model weights for the packed checkpoints.

	Qwen3.6-27B	Qwen3.8-27B	Bonsai 2 27B	
Benchmark	FP16	FP16	IQ2_XXS	Ternary
Size (GB)	53.8	53.8	7.3	5.93
Bits/weight	16.0	16.0	2.2	1.76
<i>Knowledge & reasoning</i>				
MMLU-Redux	93.26	91.46	85.79	89.09
GPQA Diamond	86.87	90.51	65.45	85.76
AA-LCR	74.00	78.00	64.00	77.00
<i>Math</i>				
GSM8K	95.60	97.19	95.38	96.66
MATH-500	99.20	99.80	94.8	98.80
AIME25	90.42	96.67	82.5	95.00
AIME26	93.33	94.58	78.6	95.83
<i>Coding</i>				
HumanEval+	95.73	93.29	87.95	95.12
MBPP+	83.07	83.86	78.31	83.07
LiveCodeBench v6	89.10	90.05	70.05	90.07
BigCodeBench	62.37	61.49	49.81	58.07
<i>Instruction following</i>				
IFEval	88.72	91.50	81.52	91.31
IFBench	60.33	71.00	52.33	74.00
<i>Agentic / tool calling</i>				
τ^2 -Bench	82.90	82.73	69.43	80.22
BFCL v3	77.19	76.74	66.1	74.92
<i>Vision</i>				
CharXiv	81.60	82.02	78.76	80.00
A-OKVQA	89.87	89.34	86.2	86.81
OmniDocBench v1.6	84.17	92.46	82.87	89.13
RealWorldQA	81.83	83.40	78.56	80.13
OCR Bench v2	61.64	60.99	55.37	56.88
Average (20)	83.6	85.4	75.2	83.9

Table 11. Full per-benchmark results (%), thinking mode with reasoning-effort `medium` for both models. Sizes are language-model weights in decimal GB. Bit-width is the average over language-model weights for the packed checkpoint.

	Qwen3.8-27B	Bonsai 2 27B
Benchmark	FP16	Ternary
Size (GB)	53.8	5.93
Bits/weight	16.0	1.76
<i>Knowledge & reasoning</i>		
MMLU-Redux	92.14	89.49
GPQA Diamond	84.34	75.56
AA-LCR	75.00	74.00
<i>Math</i>		
GSM8K	97.50	96.66
MATH-500	99.20	97.60
AIME25	86.25	74.58
AIME26	87.92	80.42
<i>Coding</i>		
HumanEval+	94.51	93.29
MBPP+	83.07	80.69
LiveCodeBench v6	83.79	80.47
BigCodeBench	63.95	59.74
<i>Instruction following</i>		
IFEval	88.35	87.99
IFBench	55.00	55.33
<i>Agentic / tool calling</i>		
τ^2 -Bench	79.50	70.86
BFCL v3	77.48	75.12
<i>Vision</i>		
CharXiv	81.58	80.02
A-OKVQA	89.78	87.51
OmniDocBench v1.6	92.48	88.85
RealWorldQA	80.00	80.00
OCR Bench v2	60.53	56.87
Average (20)	82.6	79.3

D. Throughput and Energy Measurement Notes

Throughput is reported with standardized `tg128/pp512` measurements: `tg128` is token-generation throughput over 128 generated tokens (the memory-bandwidth-bound, interactive phase), and `pp512` is prompt-processing throughput over 512 input tokens (the compute-bound phase). Both are in tokens per second, and each configuration is run repeatedly with the mean reported to reduce variance.

Measurement conditions. All throughput measurements use depth 0, batch size 1, and exclude the vision tower. Each configuration is measured three times after a warm-up pass. All rows use the same release artifact and backend build measured on 2026-09-16. Before measurement, each NVIDIA host passed a memory-bandwidth health check, reaching 76–91% of its rated bandwidth. Prefill is the noisier measurement, with run-to-run variation of up to approximately 11% on the fastest devices.

Per-platform harnesses. CUDA throughput uses the standard llama.cpp/CUDA benchmarking path; Apple laptop figures use llama-bench [31] on the Metal backend (with the custom low-bit kernels).

Energy. Power is measured with platform-native instrumentation, and the two platforms do *not* enclose the same components. On NVIDIA, `nvidia-smi` reports board power including HBM/GDDR; the mWh/token column of Table 5 is that figure, total, with no idle subtraction. On Apple, `powermetrics` reports CPU, GPU and ANE and has *no DRAM rail*, so no comparable per-token figure exists and none is quoted; the Apple measurement is reported as absolute draw in Section 3, with its idle floor stated rather than silently subtracted. Energy per token is then $E = P/(3.6r)$ in mWh, for measured power P (W) and rate r (tok/s).

Per-card decode energy for the shipped ternary packs is reported in the mWh/token columns of Table 5, measured on the current runtime; the earlier pre-rotation energy table has been dropped rather than carried forward against newer throughput figures. Prefill is far cheaper than decode per token everywhere (0.012–0.060 mWh/token, e.g. 0.017 on the RTX PRO 6000), because prompt processing is compute-bound and amortizes weight traffic across many tokens.

Architecture-Specific Throughput Behavior The two ternary packings trade memory traffic against unpacking cost. PTQ1_0 moves 18% less weight data per step than PQ2_0, but its dense trit representation requires additional arithmetic during unpacking. This produces different operating points across GPU architectures.

On the evaluated NVIDIA hardware, PTQ1_0 is faster on the Ada-generation parts and on the L4, while PQ2_0 is faster on several larger accelerators. The difference is particularly visible on H100 and A100, where the additional unpacking work outweighs the reduction in weight traffic. This reflects a shift in the limiting resource: on memory-constrained devices, reducing bytes moved is especially valuable, whereas on larger accelerators instruction throughput and launch overhead can become more important.

Prompt processing shows a different pattern because it is more compute-bound than batch-1 decoding. In the measured runs, PQ2_0 reaches 4520 tok/s on the RTX PRO 6000 and 4121 tok/s on the RTX 5090, while PTQ1_0 is substantially slower. The denser packing therefore provides its clearest advantage during bandwidth-sensitive token generation rather than compute-heavy prompt processing.

References

- [1] PrismML. 1-bit Bonsai 8B: End-to-end 1-bit language model deployment. Technical report, PrismML, March 2026.
- [2] PrismML. Bonsai 27B: binary and ternary representations of a 27B hybrid-attention model. Technical report, PrismML, July 2026.
- [3] Qwen Team. Qwen3.8-27B. <https://huggingface.co/Qwen/Qwen3.8-27B>, 2026.
- [4] Alibaba ModelScope. EvalScope: Evaluation framework for large language models. <https://github.com/modelscope/evalscope>, 2024.
- [5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSOP*, 2023.
- [6] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. MLX: An array framework for Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [7] ggml-org. llama.cpp tensor encoding schemes. <https://github.com/ggml-org/llama.cpp/wiki/Tensor-Encoding-Schemes>, 2026.
- [8] Unsloth. Qwen3.8-27B GGUF, UD-Q4_K_XL variant. <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>, 2026.
- [9] A. P. Gema et al. Are We Done with MMLU? In *NAACL*, 2025.
- [10] D. Rein et al. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022, 2023.
- [11] Artificial Analysis. Artificial Analysis Long Context Reasoning (AA-LCR). 2025. <https://artificialanalysis.ai/articles/announcing-aa-lcr>
- [12] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.

- [13] D. Hendrycks et al. Measuring Mathematical Problem Solving with the MATH Dataset. *arXiv:2103.03874*, 2021.
- [14] Y. Zhang and Math-AI Team. American Invitational Mathematics Examination (AIME) 2025. 2025. <https://huggingface.co/datasets/math-ai/aime25>
- [15] Y. Zhang and Math-AI Team. American Invitational Mathematics Examination (AIME) 2026. 2026. <https://huggingface.co/datasets/math-ai/aime26>
- [16] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *NeurIPS*, 2023.
- [17] N. Jain et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv:2403.07974*, 2024.
- [18] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv:2406.15877*, 2024.
- [19] J. Zhou et al. Instruction-Following Evaluation for Large Language Models. *arXiv:2311.07911*, 2023.
- [20] V. Pyatkin et al. Generalizing Verifiable Instruction Following. *arXiv:2507.02833*, 2025.
- [21] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv:2506.07982*, 2025.
- [22] S. G. Patil et al. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *ICML*, 2025.
- [23] Z. Wang et al. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. In *NeurIPS*, 2024.
- [24] D. Schwenk, A. Khandelwal, C. Clark, K. Marino, and R. Mottaghi. A-OKVQA: A Benchmark for Visual Question Answering using World Knowledge. *arXiv:2206.01718*, 2022.
- [25] L. Ouyang et al. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. *arXiv:2412.07626*, 2024.
- [26] xAI. RealWorldQA. <https://huggingface.co/datasets/xai-org/RealworldQA>
- [27] L. Fu et al. OCRBench v2: An Improved Benchmark for Evaluating Large Multimodal Models on Visual Text Localization and Reasoning. *arXiv:2501.00321*, 2025.
- [28] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort. SpinQuant: LLM quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024.
- [29] B. J. Fino and V. R. Algazi. Unified matrix treatment of the fast Walsh-Hadamard transform. *IEEE Transactions on Computers*, C-25(11):1142–1146, 1976.
- [30] H. Badri and A. Shaji. Half-Quadratic Quantization of large machine learning models. Mobius Labs, November 2023. https://mobiusml.github.io/hqq_blog/; code at <https://github.com/mobiusml/hqq>.
- [31] G. Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [32] Terminal-Bench Team. Terminal-Bench 2.1. <https://www.tbench.ai/news/terminal-bench-2-1>, May 2026.
- [33] N. Chowdhury, et al. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024.